SKILLS UP - BACKEND STRUCTURE

Bu hujjat backend dasturchi uchun yozilgan. Frontend (React + TypeScript) tayyor: hozir hamma ma'lumot brauzerda (localStorage va mock data) turibdi. Backend shu ma'lumotlarni va qoidalarni serverga ko'chiradi. Hujjatda: rollar, ma'lumotlar bazasi jadvallari, biznes qoidalari (aniq raqamlar bilan), hisoblash formulalari va API ro'yxati bor.

Bu hujjat asosida backend loyihasi yozilgan: papka backend/ (NestJS, qarang backend/README.md); frontend alohida papkada: frontend/. Hujjatdagi hamma raqam va qoida frontend kodidan olingan. Qoida o'zgarsa, frontend ham o'zgaradi, shuning uchun backendda ularni bitta joyda (konfiguratsiya) saqlang.

1. UMUMIY MA'LUMOT

- Tizim: o'quv markazi platformasi "Skills Up". Til: o'zbekcha (faqat logo "Skills Up" inglizcha). Xato va xabar matnlari o'zbekcha.
- Uch rol: STUDENT (o'quvchi), TEACHER (ustoz), ADMIN (administrator). Rol login va paroldan aniqlanadi.
- Pul: faqat so'm (UZS), butun son (tiyin yo'q). Masalan 500000.
- Vaqt: ISO 8601, vaqt mintaqasi Asia/Tashkent (UTC+5). Bazada UTC saqlash, javobda ISO bilan berish tavsiya etiladi.
- Dars davomiyligi: dars xonada 2 soat (jadval katagi). Dars yozuvidagi duration_min alohida (video uzunligi, 10-240 daqiqa).
- Valyuta, til va boshqa doimiy qiymatlar (XP, ball, jarima) konfiguratsiyada bo'lsin (7-bo'limga qarang).

2. TEXNOLOGIYA TAVSIYASI

Backend NestJS (TypeScript) da yoziladi. Hujjatning 1-14 bo'limlari framework'dan mustaqil (jadvallar, qoidalar, API), 15-bo'limda shu tuzilma NestJS modullariga, guardlariga, DTO va servislariga moslab berilgan. Frontend uchun faqat REST + JSON kerak:

- NestJS + TypeScript (frontend ham TypeScript: DTO va tur nomlarini bir xil qilsangiz ulash oson)
- REST API, JSON, prefiks: /api/v1
- Baza: PostgreSQL (jadvallar quyida); ORM: TypeORM yoki Prisma (o'zingiz tanlang)
- Fayl saqlash: S3-mos saqlash yoki server diski (jadval: files)
- Autentifikatsiya: JWT ISHLATILMAYDI. Login qilganda server tasodifiy token (sessiya) beradi va uning xeshini bazada saqlaydi (sessions jadvali, 5.1). Token har so'rovda `Authorization: Bearer <token>` bilan yuboriladi. JWT_SECRET kerak emas.
- Parol xeshi: argon2id yoki bcrypt (frontend demo uchun SHA-256 ishlatgan, buni ishlatmang)

3. ROLLAR VA RUXSATLAR

ROL: STUDENT

- O'z guruhlarini, darslarini, vazifalarini ko'radi.
- Dars videosini ko'radi (progress yoziladi), vazifa topshiradi (matn + fayl).
- Reyting, to'lovlar tarixi, keyingi to'lovlar, bildirishnomalar, qo'llanma/FAQ, karyera videolari.
- O'z parolini o'zgartiradi. Ism, email, telefonni o'zgartira OLMAYDI.

ROL: TEACHER

- Faqat O'Z guruhlari (groups.teacher_id = o'zi) bilan ishlaydi.
- Dars boshlaydi (imtihon qilib belgilashi mumkin), dars ma'lumotlarini (tavsif, video havola/fayl, davomiylik) kiritadi.
- Uyga vazifa beradi (matn, qadamlar, muddat, fayl), topshirilgan ishni baholaydi yoki qaytaradi.
- Davomat belgilaydi, saqlaydi (qulflaydi), qayta ochadi. Jurnalni ko'radi. Imtihon ballini kiritadi.
- O'z parolini o'zgartiradi.

ROL: ADMIN

- O'quvchi qo'shadi, guruhga joylaydi. Ustoz qo'shadi.
- O'quvchi va ustozning login va parolini o'zgartiradi.
- Xona qo'shadi, nomini va sig'imini o'zgartiradi. Xonalar jadvalini (band/bo'sh) ko'radi.
- Guruh ochadi (kunlar, 2 soatlik vaqt, xona, ustoz).
- Guruhning to'lov sanasi va summasini belgilaydi: shu guruhdagi hamma o'quvchiga bildirishnoma boradi.
- O'z parolini o'zgartiradi.

Har so'rovda server rolni va egalikni tekshirishi shart (masalan ustoz boshqa ustozning guruhiga tegolmaydi, o'quvchi boshqa o'quvchining ishini ko'rolmaydi).

4. AUTENTIFIKATSIYA

Login sahifasi: login + parol. Rol hisobga qarab aniqlanadi.

- POST /auth/login { "login": "...", "password": "..." } -> { "token": "...", "user": { id, role, fullName } }
- Login katta-kichik harfga e'tiborsiz (saqlashda kichik harfda).
- Xato bo'lsa HAR DOIM bitta umumiy javob: 401 "Login yoki parol noto'g'ri." (qaysi biri xato ekanini aytmang).
- Urinishlarni cheklang (rate limit), ketma-ket xatolardan keyin vaqtincha bloklang.
- POST /auth/logout: sessiya serverda o'chiriladi, token darhol ishlamay qoladi (204).
- POST /auth/refresh: joriy token yangisiga almashtiriladi (yangi muddat), eskisi ishlamaydi.
- Sessiya muddati SESSION_HOURS (standart 12 soat). Muddati o'tgan, noma'lum yoki o'chirilgan token -> 401.
- Parol o'zgarganda sessiyalar tugaydi: admin boshqa odamning parolini o'zgartirsa, uning HAMMA sessiyasi o'chadi; o'zi o'zgartirsa, boshqa qurilmalardagi sessiyalar o'chadi, hozirgisi qoladi.
- Login: 4-30 belgi, faqat kichik lotin harflari, raqam, nuqta, pastki chiziq, chiziq (a-z 0-9 . _ -). Butun tizimda UNIKAL (o'quvchi, ustoz, admin orasida).
- Parol: kamida 8 belgi. O'z parolini o'zgartirishda kamida bitta harf va bitta raqam bo'lishi, eski paroldan farq qilishi kerak.
- Parolni o'zgartirish (o'zi): POST /me/password { current, next } -> joriy parol noto'g'ri bo'lsa 422 "Joriy parol noto'g'ri."
- Administrator hisobi muhit o'zgaruvchilaridan boshqariladi: ADMIN_LOGIN (standart: admin) va ADMIN_PASSWORD (kamida 8 belgi). Backend har ishga tushganda bazani shularga moslaydi: admin yo'q bo'lsa yaratadi, parol farq qilsa yangilaydi (shu adminning sessiyalari tugaydi). Parol kod ichida yozilmaydi. ADMIN_PASSWORD bo'sh bo'lsa, admin hisobiga tegilmaydi.
- Admin boshqa odamning loginini/parolini o'zgartiradi (9.4-bo'lim, ADMIN).

- Rol bo'yicha yo'naltirish va rol guard: 4.1 va 4.2 bo'limlar.
- Hech qachon parol yoki xesh javobda qaytmasin.

4.1 Login -> rol -> o'z sahifasi (frontend oqimi)

Frontend login sahifasidan keyin rolga qarab O'Z panelini ochadi. Backend shu oqimni ta'minlashi kerak:

```
1. Foydalanuvchi /login da login va parolni yozadi.
2. POST /auth/login  ->  { token, user:{ id, role, fullName } }
3. Frontend role ga qarab o'tadi:
      role = STUDENT  ->  /            (o'quvchi bosh sahifasi)
      role = TEACHER  ->  /teacher     (ustoz paneli)
      role = ADMIN    ->  /admin       (admin paneli)
4. Sahifa yangilanganda frontend GET /me ni chaqiradi: token yaroqli bo'lsa role va ism olinadi, aks holda 401 -> /login.
5. "Chiqish" tugmasi: POST /auth/logout (sessiya serverda o'chadi), token o'chiriladi, /login ga qaytadi.
```

Misol: login o'quvchiniki bo'lsa (masalan "bekzod.rahimov"), kirgach O'QUVCHI paneli ochiladi va unda aynan SHU o'quvchining ma'lumotlari (guruhlari, darslari, XP, vazifalari, to'lovlari) ko'rinadi. Ustoz logini bilan kirsa ustoz paneli, admin logini bilan kirsa admin paneli ochiladi. Rolni foydalanuvchi o'zi tanlamaydi: u faqat login va paroldan keladi.

NOTA: frontend hozir (mock) o'quvchi paneli uchun bitta demo o'quvchining ma'lumotini ko'rsatadi. Backend ulanganda har o'quvchi o'zinikini ko'radi.

4.2 Rol guard (sahifalarni rol bo'yicha himoyalash)

Frontendda marshrut himoyasi (route guard) bor va quyidagicha ishlaydi:

```
Holat                                                                    Natija
-------------------------------------------------------------------------------------------------------
Tizimga kirmagan odam istalgan sahifani ochsa (/admin, /lessons...)   /login ga yo'naltiriladi
O'quvchi /admin yoki /teacher... manzilini ochsa                      o'quvchi bosh sahifasiga (/) qaytariladi
Ustoz /admin yoki o'quvchi sahifalarini (/lessons, /payments...) ochsa  ustoz bosh sahifasiga (/teacher) qaytariladi
Admin /teacher... yoki o'quvchi sahifalarini ochsa                    admin bosh sahifasiga (/admin) qaytariladi
Tizimga kirgan odam /login ni ochsa                                   o'z bosh sahifasiga o'tadi
O'z panelining ichki sahifalari                                       ochiladi
```

Rolga tegishli sahifa manzillari:

```
STUDENT   /, /lessons, /lessons/:id, /leaderboard, /career, /guides, /groups, /analytics, /payments, /settings
TEACHER   /teacher, /teacher/groups, /teacher/lessons, /teacher/lessons/new, /teacher/lessons/:id, /teacher/journal, /teacher/review, /teacher/settings
ADMIN     /admin, /admin/students(/:id), /admin/teachers(/:id), /admin/groups(/:id), /admin/rooms(/:id), /admin/payments, /admin/reports, /admin/admins, /admin/settings
```

MUHIM (backend uchun): frontend guard faqat sahifalarni brauzerda yashiradi, uni chetlab o'tish mumkin. HAQIQIY himoya backendda bo'lishi shart:

- Har so'rovda tokendan rol va foydalanuvchi aniqlansin. Rol endpoint prefiksiga mos kelmasa 403 qaytsin: /student/* faqat STUDENT, /teacher/* faqat TEACHER, /admin/* faqat ADMIN.
- O'quvchi endpointlarida o'quvchi identifikatori URL dan EMAS, tokendan olinsin (/student/... har doim "men"). Shunda o'quvchi boshqa o'quvchining ma'lumotini ko'ra olmaydi.
- Ustoz endpointlarida guruh/dars shu ustozniki ekani tekshirilsin (groups.teacher_id = token user), aks holda 403.
- Token yaroqsiz yoki muddati tugagan bo'lsa 401: frontend /login ga qaytaradi. 403 kelsa frontend foydalanuvchini o'z bosh sahifasiga qaytaradi.

5. MA'LUMOTLAR BAZASI JADVALLARI

Quyida jadvallar. Tur: uuid yoki bigint (o'zingiz tanlang), text, int, bool, timestamptz, date, time, jsonb.

5.1 Foydalanuvchilar

```
users
  id                  pk
  role                enum('STUDENT','TEACHER','ADMIN')  not null
  login               text unique not null  (kichik harf, 4-30 belgi)
  password_hash       text not null
  password_changed_at timestamptz null
  full_name           text not null           (kamida 2 so'z: ism va familiya)
  phone               text null               (format: +998 90 123 45 67)
  email               text null
  bio                 text null
  is_active           bool default true
  created_at          timestamptz default now()

students   (user_id pk fk users)  joined_at date, streak_days int default 0 (hisoblanadi, 7.9)

sessions            (kirish sessiyalari; token o'zi saqlanmaydi, faqat SHA-256 xeshi)
  id           pk
  user_id      fk users
  token_hash   text unique not null
  created_at   timestamptz
  expires_at   timestamptz not null   (created_at + SESSION_HOURS)
teachers   (user_id pk fk users)
```

Frontendda student profilida: ism, email, telefon, bio ko'rsatiladi (faqat o'qish). Faqat parol o'zgaradi.

5.2 Xonalar va guruhlar

```
rooms
  id          pk
  name        text not null  (unikal, katta-kichik harfga e'tiborsiz)
  capacity    int not null   (1..60)

groups
  id          pk
  name        text not null  (>= 3 belgi)    masalan "Fullstack dasturlash"
  course      text not null  (>= 3 belgi)    masalan "Frontend va backend asoslari"
  teacher_id  fk users(TEACHER) not null
  is_active   bool default true
  created_at  timestamptz

group_schedule      (guruhning haftalik jadvali, har kun uchun bitta qator)
  id          pk
  group_id    fk groups
  day         enum('MON','TUE','WED','THU','FRI','SAT','SUN')
  start_time  time   (08:00, 10:00, 12:00, 14:00, 16:00, 18:00, 20:00)
  end_time    time   (= start_time + 2 soat)
  room_id     fk rooms null   (NULL = onlayn, xona band qilmaydi)
  unique (room_id, day, start_time)  -- room_id NULL bo'lmaganda

group_members
  group_id    fk groups
  student_id  fk users(STUDENT)
  joined_at   timestamptz
  pk (group_id, student_id)
```

```
payment_plans       (guruhning keyingi to'lovi, har guruhga bitta)
  group_id    pk fk groups
  amount      int not null   (so'm, >= 10000)
  due_date    date not null
  updated_at  timestamptz
```

Guruh sig'imi: guruh jadvalidagi xonalarning eng kichik capacity qiymati. Faqat onlayn guruh bo'lsa 15. Sig'im to'lgan guruhga o'quvchi qo'shib bo'lmaydi.

5.3 Darslar

```
lessons
  id               pk
  group_id         fk groups
  title            text not null  (>= 3 belgi)
  description      text default ''   (bo'sh yoki >= 10 belgi)
  starts_at        timestamptz not null  (dars boshlangan sana va vaqt; ustoz "Darsni boshlash" bosganda server vaqti)
  duration_min     int default 45  (10..240)
  video_url        text null       (http:// yoki https:// bilan boshlanadi)
  video_file_id    fk files null   (video fayl; video_url bilan birgalikda bo'lmaydi: bittasi)
  is_exam          bool default false   (ustoz imtihon deb belgilagan)
  attendance_saved bool default false   (davomat saqlangan: tahrirlash yopiq)
  created_by       fk users(TEACHER)
  created_at       timestamptz
```

Muhim: dars "boshlanmagan" (rejalashtirilgan) deyiladi, agar starts_at > hozir.

```
lesson_progress     (o'quvchi videoni qancha ko'rgan)
  lesson_id     fk lessons
  student_id    fk users
  percent       int 0..100   (faqat OSHADI, hech qachon kamaymaydi)
  completed_at  timestamptz null  (percent 100 bo'lgan payt)
  updated_at    timestamptz
  pk (lesson_id, student_id)

attendance          (ustoz belgilagan davomat)
  lesson_id   fk lessons
  student_id  fk users
  present     bool not null
  marked_by   fk users(TEACHER)
  marked_at   timestamptz
  pk (lesson_id, student_id)
```

Davomat qoidasi (7.3-bo'limda batafsil): ustoz belgilamagan bo'lsa, video 100% ko'rilgan o'quvchi qatnashgan hisoblanadi. Ustoz belgisi bo'lsa, u ustun.

5.4 Uyga vazifa

```
homeworks           (ustoz bergan vazifa, har darsga bitta)
  id           pk
  lesson_id    fk lessons unique
  task         text not null  (>= 10 belgi)
  steps        jsonb not null (matnlar massivi, kamida 1 ta: "qanday bajariladi")
  assigned_at  timestamptz
  due_at       timestamptz    (assigned_at + muddat)
  time_limit_hours int        (1, 3, 24, 72, 168)

homework_attachments  (ustozning vazifa fayllari: PDF, ZIP va h.k.)
  homework_id  fk homeworks
  file_id      fk files
```

```
  pk (homework_id, file_id)

student_homework    (har o'quvchining shu vazifa bo'yicha holati)
  homework_id  fk homeworks
  student_id   fk users
  status       enum('NOT_DONE','PENDING','RETURNED','DONE')
  assigned_at  timestamptz   (qaytarilganda yangilanadi)
  due_at       timestamptz   (qaytarilganda yangilanadi: +24 soat)
  grade        int null      (1..100, faqat DONE da)
  feedback     text null     (qaytarilganda ustoz izohi)
  graded_at    timestamptz null
  graded_by    fk users null
  pk (homework_id, student_id)

submissions         (o'quvchi topshirgan ish; har urinish alohida qator, tarix saqlanadi)
  id           pk
  homework_id  fk homeworks
  student_id   fk users
  text         text default ''   (<= 2000 belgi)
  submitted_at timestamptz
  is_late      bool   (submitted_at > student_homework.due_at topshirish paytida)

submission_files
  submission_id fk submissions
  file_id       fk files
  pk (submission_id, file_id)
```

"Vazifa berilmagan" holati: darsda homeworks qatori yo'q (jadvalda NOT_ASSIGNED saqlanmaydi, API holatni hisoblab beradi).

5.5 Imtihon

```
exam_results
  lesson_id    fk lessons   (is_exam = true bo'lgan dars)
  student_id   fk users
  ball         int 1..100
  entered_by   fk users(TEACHER)
  entered_at   timestamptz
  pk (lesson_id, student_id)
```

Imtihon alohida jadval emas: imtihon = is_exam darsi. Qaysi darslarni qamrab olishi hisoblanadi (7.5).

5.6 To'lovlar va bildirishnomalar

```
invoices            (savdo/hisob: o'quvchi + guruh + davr YYYY-MM, issuedAt=savdo sanasi, dueDate, amount; unikal: studentId+groupId+period)
payments            (to'lovlar; invoiceId -> invoices; holat (PAID/UNPAID/PARTIALLY_PAID) saqlanmaydi, to'lovlardan hisoblanadi)
  id           pk
  student_id   fk users
  group_id     fk groups
  amount       int (so'm)
  paid_at      date
  method       text     masalan "Uzcard •• 8600", "Humo •• 9860"
  description  text     masalan "Standart to'lov - noyabr"

payment_notices     (admin yuborgan eslatmalar tarixi)
  id           pk
  group_id     fk groups
  amount       int
  due_date     date
```

```
  sent_at      timestamptz
  sent_by      fk users(ADMIN)
  recipients   int   (nechta o'quvchiga yuborilgan)

notifications       (har foydalanuvchining bildirishnomalari)
  id           pk
  user_id      fk users
  type         text   ('payment_notice','new_lesson','homework_graded','homework_returned','group_added',...)
  title        text
  link         text null
  created_at   timestamptz
  read_at      timestamptz null

user_settings
  user_id        pk fk users
  notify_email   bool default true
  notify_push    bool default true
  notify_reminders bool default true
  theme          enum('light','dark','system') default 'dark'   (ixtiyoriy; hozir brauzerda)
```

To'lov (payment) uchun ball, XP yoki kumush BERILMAYDI.

5.7 Fayllar va kontent

```
files
  id            pk
  owner_id      fk users
  original_name text
  mime_type     text
  size_bytes    bigint
  storage_key   text   (diskdagi yoki S3 dagi joy)
  created_at    timestamptz

career_videos   id, title, topic, duration_min, description, video_url   (faqat ko'rish; progress va mukofot yo'q)
guides          id, title, category, read_min, summary, body
faqs            id, question, answer
activities      id, student_id, type('lesson','badge','rank','payment','group'), text, xp int null, at timestamptz   (bosh sahifadagi "so'nggi faoliyat")
calendar_events (ixtiyoriy: darslar, imtihonlar, vazifa muddatlari, suhbatlar mavjud ma'lumotdan hisoblanadi)
```

6. BIZNES QOIDALARI (ANIQ RAQAMLAR)

6.1 Konfiguratsiya qiymatlari

```
ATTENDANCE_XP        = 20     darsga qatnashish uchun XP (hamma darsda bir xil)
ATTENDANCE_GEMS      = 10     darsga qatnashish uchun kumush
MIN_BALL / MAX_BALL  = 1 / 100   vazifa va imtihon bali (butun son)
XP_PER_BALL          = 1      1 ball = 1 XP
BALL_PER_GEM         = 2      har 2 ball = 1 kumush (yaxlitlash: Math.round(ball / 2))
LATE_PENALTY         = 10     kechikib topshirilsa, balldan ayiriladigan ball
XP_PER_LEVEL         = 250    bosqich (level) uchun XP
HOMEWORK_LIMITS      = 1, 3, 24, 72, 168 soat   (1 soat, 3 soat, 1 kun, 3 kun, 1 hafta)
RETURN_EXTENSION_H   = 24     qaytarilgan ishga beriladigan yangi muddat (soat)
LESSON_SLOTS         = 08:00, 10:00, 12:00, 14:00, 16:00, 18:00, 20:00 (har biri 2 soat)
DEFAULT_ONLINE_CAPACITY = 15
```

6.2 Dars va video progress

- O'quvchi videoni ko'radi. Progress = videoning ENG ILGARI ko'rilgan joyi (foizda). Oldinga o'tkazib yuborish bilan progress oshmaydi, faqat o'ynatib ko'rish bilan oshadi (frontend buni ta'minlaydi, backend faqat oshishga ruxsat bersin: yangi qiymat >= eskisi).
- Progress 100 bo'lsa, dars "tugatilgan" (completed), aks holda "jarayonda" (in-progress). Birinchi ko'rishdan 100% gacha davom ettirish mumkin (qaldirgan joyidan).
- DARSLARGA KIRISH CHEKLANMAGAN: oldingi darsning vazifasi topshirilmagan bo'lsa ham o'quvchi keyingi darsni ochadi va qatnashadi. Vazifa faqat IMTIHONga kirishga ta'sir qiladi (6.6).
- Karyera videolari: faqat ko'rish, progress yozilmaydi.

6.3 Davomat

- Ustoz har o'quvchini "qatnashdi" yoki "qatnashmadi" deb belgilaydi (dars boshlangan bo'lsa; rejalashtirilgan darsga davomat qo'yib bo'lmaydi).
- Ustoz belgisi YO'Q bo'lsa: video progress 100 bo'lsa qatnashgan hisoblanadi.
- Ustoz belgisi BOR bo'lsa: shu belgi ustun (video ko'rilgan-ko'rilmaganiga qaramay).
- "Hammasi qatnashdi" tugmasi: hali qatnashmagan hammani present = true qiladi.
- "Davomatni saqlash": lessons.attendance_saved = true. Shu paytda hisoblangan natija HAR BIR o'quvchi uchun attendance jadvaliga aniq (explicit) yozilsin (video asosida hisoblanganlar ham). Saqlangandan keyin davomatni tahrirlash endpointlari 409 qaytaradi ("Davomat saqlangan").
- "Davomatni tahrirlash": attendance_saved = false (faqat ustoz). Jurnalda saqlangan darslar qulf belgisi bilan ko'rinadi, katak o'zgarmaydi.
- Qatnashish uchun mukofot: har doim ATTENDANCE_XP va ATTENDANCE_GEMS.

6.4 Uyga vazifa

Berish (ustoz):

- Matn: kamida 10 belgi. Qadamlar: kamida 1 ta (har biri alohida qator, bo'sh emas).
- Muddat: HOMEWORK_LIMITS dan biri; due_at = hozirgi vaqt + muddat; assigned_at = hozir.
- Fayl: ixtiyoriy, 5 tagacha, har biri 20 MB gacha (8-bo'lim).
- Vazifa berilganda shu guruhning hamma o'quvchisi uchun student_homework NOT_DONE bo'lib yaratiladi.
- Vazifani YANGILASH (matn, qadam, muddat, fayl) faqat hali topshirmaganlarga ta'sir qiladi: ya'ni holati "berilmagan" yoki NOT_DONE bo'lganlarga. Topshirgan (PENDING), qaytarilgan (RETURNED) yoki baholangan (DONE) o'quvchilar o'z holati va muddatini saqlab qoladi.
- Dars boshlangandan KEYIN guruhga qo'shilgan o'quvchi uchun ham student_homework yaratilsin.

Holatlar (o'quvchi uchun) va o'tishlar:

```
berilmagan (qator yo'q)  --ustoz vazifa beradi-->  NOT_DONE
NOT_DONE   --o'quvchi topshiradi-->                PENDING
PENDING    --ustoz baholaydi (ball 1..100)-->      DONE
PENDING    --ustoz qaytaradi (izoh bilan)-->       RETURNED   (assigned_at = hozir, due_at = hozir + 24 soat, grade o'chiriladi)
RETURNED   --o'quvchi qayta topshiradi-->          PENDING
DONE       --ustoz bahoni o'zgartiradi-->          DONE (yangi ball) yoki RETURNED
```

Frontendda holat nomlari: Bajarilgan (DONE), Kutilmoqda (PENDING), Qaytarilgan (RETURNED), Bajarilmagan (NOT_DONE), Berilmagan.

Topshirish (o'quvchi):

- Faqat NOT_DONE yoki RETURNED holatda. Muddat o'tgan bo'lsa ham topshirish MUMKIN, lekin kechikkan hisoblanadi.
- Matn (<= 2000 belgi) VA/YOKI fayl: kamida bittasi bo'lishi shart ("Javob yozing yoki kamida bitta fayl biriktiring.").

- Fayl: 3 tagacha, har biri 5 MB gacha (8-bo'lim). Papka qabul qilinmaydi.
- is_late = (topshirish vaqti > student_homework.due_at).
- Topshirgach holat PENDING.

Baholash (ustoz): ball butun son, 1..100. Ustoz ball qo'yadi (grade), holat DONE. Qaytarish: izoh (feedback) majburiy.

6.5 Kechikish jarimasi

- Ustoz qo'ygan ball (raw_ball) o'zgarmaydi, bazada shunday saqlanadi.
- Hisoblashda: agar ish kechikib topshirilgan bo'lsa (oxirgi topshirish is_late = true), hisobga olinadigan ball = max(1, raw_ball - 10).
- XP va kumush shu hisobga olinadigan balldan hisoblanadi.
- Interfeysda ikkala qiymat ko'rsatiladi: "85 - 10 = 75".

6.6 Imtihon

- Imtihon = ustoz darsni boshlayotganda "Bu dars imtihon" ni yoqqan dars (lessons.is_exam = true). 13 darsdan keyin degan qat'iy qoida YO'Q: ustoz xohlagan paytda belgilaydi.
- Imtihon guruh ichida sana bo'yicha tartiblanadi (1-imtihon, 2-imtihon...).
- Imtihon qamrab oladigan darslar: shu guruhning oldingi imtihondan (yoki guruh boshidan) beri o'tilgan oddiy (is_exam = false) darslari.
- Imtihonga KIRISH SHARTI: qamrab olingan darslarning vazifasi topshirilgan bo'lishi kerak. Kamchilik (missing) deb quyidagilar hisoblanadi: shu o'quvchi uchun vazifa berilgan, holati NOT_DONE VA muddati (due_at) o'tib ketgan. Muddati hali o'tmagan NOT_DONE vazifa kamchilik emas.
- Ruxsat bor o'quvchiga ustoz ball (1..100) kiritadi. Ruxsati yo'q o'quvchiga ball kiritish MUMKIN EMAS (backend 409 qaytarsin).
- Imtihon bali ham XP va kumushga shu qoidada aylanadi (1 ball = 1 XP, 2 ball = 1 kumush). Imtihon uchun kechikish jarimasi yo'q.
- Imtihon darsining o'zida davomat ham bor (oddiy dars kabi).
- API javobida har o'quvchi uchun imtihon holati: "done" (ball bor), "ready" (ruxsat bor, ball yo'q), "blocked" (ruxsat yo'q + missing darslar ro'yxati).

6.7 Guruh, xona va jadval qoidalari (admin)

- Dars vaqtlari 2 soatlik katak: 08:00, 10:00, 12:00, 14:00, 16:00, 18:00, 20:00 (tugashi +2 soat).
- Xona bandligi: (xona, hafta kuni, boshlanish vaqti) bo'yicha UNIKAL. Bir xonada bir vaqtda ikkita guruh bo'lolmaydi.
- Yangi guruh ochish: kamida 1 kun, bitta 2 soatlik vaqt, xona. Tanlangan xona BARCHA tanlangan kunlarda shu vaqtda bo'sh bo'lishi kerak.
- Ustoz to'qnashuvi: ustoz shu kun va shu boshlanish vaqtida boshqa guruhda dars o'tmasligi kerak. Xato xabari: "Ustoz shu vaqtda “<guruh>” guruhida dars o'tadi."
- Xona band bo'lsa xato: "<xona> <kun> kuni <vaqt> da band: <guruh>."
- Onlayn mashg'ulot xona band qilmaydi (room_id NULL).
- Xonani qo'shish: nom unikal (katta-kichik harfga e'tiborsiz), sig'im 1..60. Xona nomini o'zgartirsa, guruh jadvalida yangi nom ko'rinadi (jadval room_id orqali bog'langan).
- Yangi guruh ochilganda: group_schedule qatorlari (har kun uchun) + payment_plans qatori (summa >= 10000, birinchi to'lov sanasi bugundan oldin bo'lmasin).
- O'quvchini guruhga qo'shish: sig'im to'lmagan bo'lishi, o'quvchi bu guruhda allaqachon bo'lmasligi kerak. Xato: "<guruh> guruhida bo'sh joy qolmagan." / "O'quvchi bu guruhda allaqachon bor."

6.8 To'lov eslatmasi (admin -> o'quvchilar)

- Admin guruh uchun keyingi to'lov summasi va sanasini belgilaydi va "Saqlash va yuborish" bosadi.
- Server: payment_plans yangilanadi, payment_notices ga qator qo'shiladi, guruhning HAR BIR hozirgi o'quvchisi uchun notifications qatori yaratiladi (type = 'payment_notice').
- Bildirishnoma matni: "To'lov eslatmasi: keyingi to'lov 20-dekabr gacha, 500 000 so'm".
- Guruhda o'quvchi bo'lmasa: xato "Guruhda o'quvchi yo'q: avval o'quvchi qo'shing."
- O'quvchining "To'lovlarim" sahifasi: o'z guruhlarining payment_plans qatorlari "Keyingi to'lovlar" sifatida (guruh nomi, sana, necha kun qoldi, summa) + o'tgan to'lovlar tarixi.

7. HISOBLASH FORMULALARI (SERVER HISOBLASIN)

XP, kumush, bosqich, reyting bazada saqlanmaydi, qatnashish va ball ma'lumotidan hisoblanadi (yoki keshlanadi). Frontend ularni mana shunday hisoblaydi.

7.1 Bitta dars uchun o'quvchi mukofoti

```
attended      = ustoz belgisi bor ? present : (lesson_progress.percent == 100)
attendanceXp  = attended ? 20 : 0
attendanceGem = attended ? 10 : 0
raw_ball      = student_homework.status == 'DONE' ? grade : null
ball          = raw_ball == null ? null : (is_late ? max(1, raw_ball - 10) : raw_ball)
ballXp        = ball * 1
ballGems      = round(ball / 2)
lessonXp      = attendanceXp + ballXp
lessonGems    = attendanceGem + ballGems
```

7.2 Imtihon mukofoti

```
examXp   = ball * 1
examGems = round(ball / 2)
```

7.3 Jami XP va bosqich

```
totalXp   = sum(lessonXp over all lessons) + sum(examXp)    (o'quvchi barcha guruhlari bo'yicha)
level     = floor(totalXp / 250) + 1
levelFrom = (level - 1) * 250
levelTo   = level * 250
percent   = round((totalXp - levelFrom) / 250 * 100)
remaining = levelTo - totalXp
```

7.4 Guruh reytingi

- Har guruh uchun alohida reyting. O'quvchining XP si SHU guruh darslari va shu guruh imtihonlaridan yig'iladi (boshqa guruhdagi XP kirmaydi).
- Tartib: XP kamayish bo'yicha. Teng bo'lsa barqaror tartib (masalan ism bo'yicha). rank = 1,2,3...
- delta: o'tgan haftaga nisbatan o'rin o'zgarishi (+ ko'tarilgan, - tushgan). Buning uchun haftalik snapshot (jadval rank_snapshots: group_id, student_id, week, rank) kerak yoki delta ni hozircha 0 qaytaring.
- Umumiy reyting (bosh sahifa): hamma o'quvchilar orasida totalXp bo'yicha o'rin va jami o'quvchilar soni.

7.5 Imtihon qamrovi va ruxsat (algoritm)

```
guruh darslarini sana (va vaqt) bo'yicha tartibla
block = []
har dars uchun:
   agar dars.is_exam:  imtihon = { id = dars.id, number = (nechanchi), lessons = block }; block = []
   aks holda:          block.push(dars.id)

o'quvchi uchun imtihonga ruxsat:
   missing = imtihon.lessons ichida shunday darslar: vazifa berilgan VA student_homework.status == NOT_DONE VA due_at < hozir
   ruxsat = (missing bo'sh)
```

"Oxirgi imtihondan beri o'tilgan darslar soni" = oxirgi imtihon darsidan keyingi darslar soni (ustoz va o'quvchi interfeysida ko'rsatiladi).

7.6 Ustoz statistikasi (guruh bo'yicha, har o'quvchi)

Qaytariladi: rank, xp, gems, avgBall (baholangan ishlarning o'rtacha bali, yo'q bo'lsa null), held (o'tilgan darslar soni), attended (qatnashgan), assigned (berilgan vazifa), done (baholangan), pending (tekshirishni kutayotgan).

7.7 Jurnal (davomat jadvali)

Guruh bo'yicha: o'quvchilar x darslar. Har katak: "present", "absent" yoki "upcoming" (dars hali o'tilmagan). Har is_exam darsdan keyin imtihon ustuni (done/ready/blocked + ball). Har o'quvchi uchun: o'tilgan darslar soni, qatnashgan soni, foiz = round(attended / held * 100). Har dars uchun attendance_saved bayrog'i (qulf belgisi). Saqlangan dars katagi o'zgartirilmaydi.

7.8 O'quvchi bosh sahifa va tahlil

- Bosh sahifa: totalXp, gems, level (7.3), kunlik seriya (streak), guruhdagi o'rin(lar), oxirgi faoliyat (activities), yaqin hodisalar (darslar, imtihonlar, vazifa muddatlari).
- Kunlik seriya (streak): ketma-ket nechta kun o'quvchi faol bo'lgan (video ko'rgan, vazifa topshirgan yoki darsga qatnashgan). Bugun faol bo'lmasa seriya 0 ga tushadi (yoki kechagi kungacha hisoblanadi, backend qoidasini belgilasin).
- Tahlil (analytics): weeklyHours (haftalik o'qish soatlari), xpByWeek (haftalik XP), skills (ko'nikmalar foizi), completionRate (tugatilgan darslar foizi), totalHours, lessonsCompleted, avgScore (o'rtacha ball). Hammasi darslar/progress/ball dan hisoblanadi; ko'nikmalar (skills) uchun darsga teg (topic) kerak bo'lishi mumkin: ochiq savol (13-bo'lim).

8. FAYL YUKLASH QOIDALARI

Faqat FAYL qabul qilinadi, PAPKA emas (papkani ZIP qilib yuborish kerak). Server kengaytma va mime turini tekshirsin, hajmni cheklasin, viruslarga tekshirish tavsiya etiladi.

Maqsad	Soni	Hajm (har biri)	Ruxsat etilgan kengaytmalar
O'quvchi javobiga biriktirish	3 tagacha	5 MB	.pdf .zip .doc .docx .txt .md .png .jpg .jpeg .html .css .js .ts .tsx .jsx .json
Ustozning vazifa materiali	5 tagacha	20 MB	.pdf .zip .rar .7z .doc .docx .ppt .pptx .xls .xlsx .txt .md .png .jpg .jpeg .json .html .css .js .ts .tsx
Dars videosi (fayl)	1 ta	300 MB	.mp4 .webm .mov .m4v

- Dars videosi uchun havola (video_url, http/https) YOKI fayl: bittasi.
- Video fayl oqim (range request) bilan o'ynalishi kerak, chunki o'quvchi progressi videoga qarab yuradi.
- Frontendda hozir fayllar serverga yuborilmaydi (faqat nom va hajm saqlanadi); backend tayyor bo'lgach haqiqiy yuklash ulanadi.
- Yuklash oqimi tavsiyasi: avval POST /files (multipart) -> { id, name, size }; keyin shu id lar vazifa/javob/dars so'rovida beriladi.

9. API (REST, /api/v1)

Belgilar: [S] STUDENT, [T] TEACHER, [A] ADMIN, [*] hamma rol. Hamma so'rov (login dan tashqari) token talab qiladi. Ro'yxat javoblari uchun sahifalash tavsiya: ?page=1&limit=50.

9.1 Umumiy

```
POST   /auth/login                { login, password }                    -> { token, user }          [public]
POST   /auth/logout                                                      -> 204                      [*]
GET    /me                                                               -> { id, role, fullName, login, phone, email, bio }   [*]
POST   /me/password               { current, next }                      -> 204                      [*]
GET    /me/settings                                                      -> { notifyEmail, notifyPush, notifyReminders, theme }  [*]
PUT    /me/settings               { notifyEmail?, notifyPush?, notifyReminders?, theme? }  -> settings   [*]
GET    /notifications             ?unread=true                           -> [ { id, type, title, link, createdAt, readAt } ]   [*]
POST   /notifications/read-all                                           -> 204                      [*]
POST   /files                     multipart file                         -> { id, name, size, mime } [*]
GET    /files/:id                                                        -> fayl (yuklab olish)      [egasi yoki guruh a'zolari]
```

9.2 O'quvchi [S]

```
GET  /student/dashboard
     -> { xp, gems, level, levelPercent, levelRemaining, streak, overallRank, totalStudents,
          groupRanks:[{groupId,groupName,rank,size}], activities:[...], upcoming:[...] }

GET  /student/groups
     -> [ { id, name, course, teacher:{id,fullName}, members, schedule:[{day,start,end,room}], nextSession } ]

GET  /student/groups/:groupId/lessons        ?order=newest|oldest  &status=  &homework=  &q=
     -> { group, stats:{progress, avgBall, graded, attended, held},
          exams:[ {id, number, date, title, status:'done'|'ready'|'blocked', ball?, missingLessons?:[...]} ],
          lessons:[ { id, title, startsAt, durationMin, status:'completed'|'in-progress', progress, isExam,
                      attendance:'present'|'absent'|null,
                      homework:{ status:'done'|'pending'|'returned'|'not-done'|'not-assigned', dueAt?, ball?, rawBall?, late? } } ] }

GET  /student/lessons/:lessonId
     -> { lesson (title, description, startsAt, durationMin, isExam), video:{ url? | fileId? }, progress,
          reward:{ attended, attendanceXp, attendanceGems, ball, rawBall, late, penalty, ballXp, ballGems },
          homework:{ status, task, steps:[...], attachments:[{id,name,size}], assignedAt, dueAt, timeLimit,
                     feedback?, submission?:{ text, files:[...], submittedAt, late } },
          otherLessons:[ ...shu guruhning boshqa darslari qisqacha ] }

PUT  /student/lessons/:lessonId/progress     { percent }      -> { progress, completed }     (faqat oshadi)
POST /student/lessons/:lessonId/homework     { text?, fileIds:[...] }   -> submission        (422: bo'sh javob, ko'p fayl...)

GET  /student/groups/:groupId/leaderboard    -> [ { rank, studentId, name, xp, delta, isMe } ]
GET  /student/payments                       -> { total, count, last, upcoming:[{groupId,groupName,dueDate,amount,daysLeft}], items:[{id,date,groupName,description,method,amount}] }
GET  /student/analytics                      -> { weeklyHours, xpByWeek, skills, completionRate, totalHours, lessonsCompleted, avgScore }
GET  /student/calendar                       ?month=2026-11   -> [ { id, date, time, title, type:'lesson'|'exam'|'deadline'|'interview' } ]
GET  /career-videos                          -> [ { id, title, topic, durationMin, description, url } ]
GET  /guides   GET /faqs                     -> kontent ro'yxati
```

9.3 Ustoz [T] (faqat o'z guruhlari)

```
GET  /teacher/dashboard                      -> { groups, waitingReview, upcomingLessons, stats }
GET  /teacher/groups                         -> [ { id, name, course, members, schedule, waitingCount } ]
GET  /teacher/groups/:groupId/students       -> reyting va statistika (7.6)
```

```
GET  /teacher/groups/:groupId/lessons        ?order=&q=
     -> { lessons:[ { id,title,startsAt,isExam,upcoming, summary:{students,attended,handedIn,pending,done,returned,notDone}, homework:{given,dueAt,timeLimit} } ],
          exams:[ { id, number, date, title, admitted, total, graded } ], sinceLastExam: 5 }

POST /teacher/lessons                        { groupId, title, isExam? }    -> lesson      (starts_at = server vaqti; duration_min = 45; tavsif bo'sh)
GET  /teacher/lessons/:lessonId              -> { lesson, video, homework, attendanceSaved, students:[ { studentId, name, attended, homework:{status, submittedAt, late, ball, rawBall},
    canReview } ], sameGroupLessons:[...] }
PUT  /teacher/lessons/:lessonId/info         { description?, videoUrl?|videoFileId?, durationMin }    -> lesson
PUT  /teacher/lessons/:lessonId/homework     { task, steps:[...], limitHours, attachmentIds:[...] }   -> homework   (bergan yoki yangilash; 6.4)

PUT  /teacher/lessons/:lessonId/attendance/:studentId   { present }       -> 204   (attendance_saved bo'lsa 409)
POST /teacher/lessons/:lessonId/attendance/mark-all                      -> 204   (hammasi qatnashdi)
POST /teacher/lessons/:lessonId/attendance/save                          -> 204   (saqlash: qulflash; 6.3)
POST /teacher/lessons/:lessonId/attendance/reopen                        -> 204   (tahrirlash uchun qayta ochish)

GET  /teacher/lessons/:lessonId/students/:studentId/submission   -> { text, files, submittedAt, late, status, grade?, feedback? }
POST /teacher/lessons/:lessonId/students/:studentId/grade        { ball }              -> 204   (1..100)
POST /teacher/lessons/:lessonId/students/:studentId/return       { feedback }          -> 204   (yangi muddat +24 soat)

GET  /teacher/review                         ?groupId=  -> tekshirishni kutayotgan ishlar (PENDING) ro'yxati
GET  /teacher/groups/:groupId/journal        -> { columns:[lesson|exam], rows:[ { studentId,name,cells,examCells,held,attended,percent } ] }
GET  /teacher/exams/:examId                  -> { exam, students:[ { studentId,name,state:'done'|'ready'|'blocked',ball?,missingLessons? } ] }
PUT  /teacher/exams/:examId/students/:studentId   { ball }   -> 204    (blocked bo'lsa 409; ball 1..100)
```

9.4 Admin [A]

```
GET   /admin/dashboard        -> { studentsCount, teachersCount, groupsCount, freeSlotsToday, upcomingPayments:[...], lastNotices:[...] }

# O'quvchilar  (tahrirlash va o'chirish ham bor)
GET   /admin/students         ?q=&groupId=         -> [ { id, fullName, phone, login, groups:[...], passwordChangedAt, isNew } ]
POST  /admin/students         { fullName, phone, login, password, groupIds:[...] }   -> student
         tekshiruvlar: ism >= 2 so'z; telefon +998 XX XXX XX XX; login unikal va 4-30 belgi; parol >= 8; guruh sig'imi
POST  /admin/students/:id/enrol   { groupId }                                    -> 204   (sig'im, takror tekshiruvi)
PUT   /admin/students/:id/account { login?, password? }                          -> 204   (login unikal; parol xeshlanadi)

PUT   /admin/students/:id         { fullName?, phone? }                          -> 204
DELETE /admin/students/:id        hisobni o'chiradi (isActive=false), guruhlardagi joyi bo'shaydi -> 204
DELETE /admin/students/:id/groups/:groupId   guruhdan chiqarish                   -> 204

# Ustozlar
GET   /admin/teachers         -> [ { id, fullName, phone, login, groups:[...], passwordChangedAt } ]
POST  /admin/teachers         { fullName, phone, login, password }               -> teacher
PUT   /admin/teachers/:id         { fullName?, phone? }                          -> 204
DELETE /admin/teachers/:id        faol guruhi bo'lsa 409; bo'lmasa hisob o'chiriladi   -> 204
PUT   /admin/teachers/:id/account { login?, password? }                          -> 204

# Adminlar
GET   /admin/admins               -> [ { id, fullName, phone, login, passwordChangedAt, isNew } ]
POST  /admin/admins               { fullName, phone?, login, password }           -> admin
PUT   /admin/admins/:id           { fullName?, phone? }                          -> 204
PUT   /admin/admins/:id/account   { login?, password? }                          -> 204 (parol o'zgarsa, uning sessiyalari tugaydi)
DELETE /admin/admins/:id          o'zini va oxirgi adminni o'chirib bo'lmaydi (409)    -> 204

# Xonalar
GET   /admin/rooms            -> [ { id, name, capacity } ]
POST  /admin/rooms            { name, capacity }                                 -> room   (nom unikal, sig'im 1..60)
PUT   /admin/rooms/:id        { name, capacity }                                 -> room
DELETE /admin/rooms/:id       faol guruh ishlatayotgan xonani o'chirib bo'lmaydi (409)  -> 204
GET   /admin/rooms/timetable  ?day=MON         -> { slots:["08:00",...], rooms:[ { id,name,capacity, cells:{ "08:00": null | {groupId,groupName,teacher} } } ] }
```

```
GET   /admin/rooms/free       ?days=MON,WED&start=18:00   -> [ roomId, ... ]   (hamma ko'rsatilgan kunlarda bo'sh xonalar)

# Guruhlar
GET   /admin/groups           -> [ { id,name,course,teacher,members,capacity,schedule:[{day,start,end,roomId,roomName}],plan:{amount,dueDate} } ]
POST  /admin/groups           { name, course, teacherId, days:['MON','WED'], start:'18:00', roomId, amount, dueDate }   -> group
PUT   /admin/groups/:id       { name?, course?, teacherId?, days?, start?, roomId? }  -> group  (jadval: kunlar+vaqt+xona birga; band bo'lsa 409)
DELETE /admin/groups/:id      tugagan guruhni BUTUNLAY o'chiradi (bitta tranzaksiya): darslar, davomat, video progress, imtihon ballari,
                              uyga vazifa va barcha javoblar, yuklangan fayllar (bazadan ham, R2/diskdan ham), hisoblar, to'lovlar,
                              to'lov xabarlari, jadval, o'rinlar. XP/kumush darslardan hisoblangani uchun guruhdagi XP ham yo'qoladi
                              (boshqa guruhlardagisi qoladi). O'quvchi/ustoz hisoblari o'chmaydi.
                              -> { result:'deleted', removed:{ lessons, students, invoices, payments, files } }
        tekshiruvlar: 6.7 (xona bandligi, ustoz to'qnashuvi, amount >= 10000, dueDate >= bugun)

# To'lovlar
GET   /admin/payments         -> { groups:[ { groupId,name,members,plan:{amount,dueDate},daysLeft } ], notices:[ ...tarix ] }
PUT   /admin/groups/:id/payment-plan   { amount, dueDate }   -> { plan, notice:{ id, recipients } }
         guruhdagi hamma o'quvchiga bildirishnoma yuboradi (6.8)
POST  /admin/invoices         { groupId, dueDate, period?(YYYY-MM), amount?, issuedAt? } -> { group, period, amount, created, skipped }
                              guruhdagi HAR bir o'quvchiga bitta hisob; avval yozilganlar o'tkazib yuboriladi (ikki marta hisoblanmaydi)
POST  /admin/invoices/:id/payments { amount, paidAt?, method, description? } -> hisob qatori (qolgan qarzdan ko'p bo'lsa 422)

--- To'lovlar va savdo hisoboti (faqat ADMIN; ustoz/o'quvchi 403, tokensiz 401) ---
GET   /admin/reports/payments?from&to&q&status&groupId&page&limit
      from va to BIRGA (YYYY-MM-DD), q = mijoz ismi, status = PAID|UNPAID|PARTIALLY_PAID, limit <= 100
      -> { filters, summary:{ totalSales, paid, debt, unpaidAmount, partialPaid, partialRemaining, paidInFull, overdueDebt,
                              paymentPercent, invoices, counts:{PAID,UNPAID,PARTIALLY_PAID} },
           comparison:{ previousFrom, previousTo, previousTotalSales, previousPaid, growthPercent|null, paidGrowthPercent|null } | null,
           series:{ bucketDays, points:[{ label, previousLabel, current, previous, paid }] } | null,
           rows:[{ id, studentId, studentName, groupId, groupName, period, issuedAt, dueDate, total, paid, debt, status, overdue }],
           page, limit, totalRows, totalPages }
      Hisoblash: total = paid + debt (har qatorda va jamida); paid = to'liq to'langanlar + qisman to'langanlarning to'langan qismi;
      debt = to'lanmagan + qisman to'langanlarning qolgan qismi. Hech narsa ikki marta sanalmaydi.
      Oldingi davr = xuddi shu uzunlikdagi, from dan oldingi davr. Oldingi davrda savdo yo'q bo'lsa growthPercent = null.
      Sana oralig'i berilmasa comparison va series = null.
GET   /admin/reports/payments/export?(xuddi shu filtrlar)   -> HAQIQIY .xlsx (Content-Disposition: attachment)
      Varaqlar: Summary (filtrlar, ko'rsatkichlar, oldingi davr bilan o'sish), Payments (BARCHA mos hisoblar, sahifalashsiz, JAMI qatori),
      Transactions (har bir to'lov). Kutubxona: exceljs.

--- Kontent (faqat ADMIN; o'qish hamma uchun: GET /career-videos, /guides, /faqs) ---
POST|PUT /admin/content/career-videos[/:id]  { title, topic, durationMin, description, url? (http/https, YouTube ham)}  DELETE /:id -> 204
POST|PUT /admin/content/guides[/:id]         { title, category, readMin, summary, body }                              DELETE /:id -> 204
POST|PUT /admin/content/faqs[/:id]           { question, answer }                                                     DELETE /:id -> 204

--- "Bitta yozuvni ochish" sahifalari (bog'langan yozuvlar id + nom bilan; one-to-many va many-to-many uchun) ---
GET   /admin/teachers/:id     -> { ...shaxs, groups:[guruh+jadval+o'quvchilar+darslar], totals:{ groups, archivedGroups, students, weeklySessions, lessons } }
GET   /admin/students/:id     -> { ...shaxs, groups:[guruh+ustoz+jadval], invoices:[hisob qatorlari], payments:[...], totals:{summary} }
GET   /admin/groups/:id       -> { ...guruh, teacher:{id,fullName}, lessonsHeld, totals, recentInvoices,
                                   students:[{ id, fullName, phone, login, isActive, createdAt, joinedAt, otherGroups:[{id,name}],
                                               xp, gems, level, rank, attendance:{attended,lessons,percent},
                                               homework:{given,done,pending,returned,notDone,averageGrade}, exams:{count,averageBall},
                                               bills:[hisob qatorlari], ...summary (jami/to'langan/qarz/holatlar soni) }] }
GET   /admin/rooms/:id        -> { id, name, capacity, weeklySlots, groups:[{id,name,teacher,members,slots}] }
      (POST /admin/payments olib tashlandi: to'lov endi hisobga bog'lanadi.)

--- To'lov xabarlari (saytda to'lov tizimi yo'q: o'quvchi kartadan o'zi to'laydi, summani yozadi, admin tasdiqlaydi) ---
payment_claims jadvali: invoiceId, studentId, amount, method, status PENDING|APPROVED|REJECTED, rejectReason, decidedById, decidedAt
GET   /student/payments       -> endi invoices (faqat admin yuborgan, qarzi bor hisoblar + pendingClaim), claims, items, total...
POST  /student/invoices/:id/claims { amount, method? } -> { id, status:'PENDING', amount }   (faqat o'z hisobi; qarzdan ko'p bo'lmasin;
                              bitta hisobga bitta ochiq xabar; pul tasdiqlanmaguncha HISOBLANMAYDI)
```

```
GET   /admin/claims?status=PENDING|APPROVED|REJECTED
POST  /admin/claims/:id/approve   -> haqiqiy Payment yaratiladi (tranzaksiya, qolgan qarz qayta tekshiriladi), o'quvchiga bildirishnoma
POST  /admin/claims/:id/reject    { reason? } -> o'quvchi sababni ko'radi
POST /admin/invoices endi hisob yaratilganda guruh o'quvchilariga bildirishnoma ham yuboradi (to'lov shundan keyin ochiladi).

Eski to'lovlar (invoiceId yo'q) server ishga tushganda avtomatik hisobga ko'chiriladi: har bir o'quvchi/guruh/oy uchun bitta hisob,
summasi to'langan summaga teng (PAID).
```

10. XATOLAR VA VALIDATSIYA

Xato javobi (hamma xato uchun bir xil):

```
HTTP 4xx/5xx
{ "error": { "code": "VALIDATION_ERROR", "message": "O'zbekcha xabar", "fields": { "login": "Bunday login band." } } }
```

Kodlar: 400 noto'g'ri so'rov, 401 tizimga kirilmagan/login xato, 403 ruxsat yo'q (rol yoki egalik), 404 topilmadi, 409 ziddiyat (davomat saqlangan, xona band, ruxsati yo'q o'quvchiga imtihon bali), 413 fayl katta, 415 fayl turi mos emas, 422 validatsiya, 429 juda ko'p urinish.

Validatsiya qoidalari (frontend bilan bir xil bo'lsin, serverda ham tekshiring):

```
ism-familiya       kamida 2 so'z, kamida 5 belgi
telefon            +998 90 123 45 67  (probel yoki chiziq bo'lishi mumkin; saqlashda +998 90 123 45 67 ko'rinishiga keltiring)
login              ^[a-z0-9._-]{4,30}$ , unikal
parol              >= 8 belgi
guruh nomi/yo'nalish  >= 3 belgi
dars nomi          >= 3 belgi
dars tavsifi       bo'sh yoki >= 10 belgi
dars davomiyligi   butun son 10..240
video havola       ^https?://\S+$
vazifa matni       >= 10 belgi
vazifa qadamlari   kamida 1 ta, bo'sh emas
ball               butun son 1..100
xona sig'imi       butun son 1..60
to'lov summasi     butun son >= 10000 (so'm)
to'lov sanasi      bugundan oldin bo'lmasin
javob matni        <= 2000 belgi
```

Xabar matnlari (frontenddagi misollar): "Login yoki parol noto'g'ri.", "Bunday login band. Boshqa login tanlang.", "Joriy parol noto'g'ri.", "Davomat saqlangan.", "Bunday nomli xona bor.", "<guruh> guruhida bo'sh joy qolmagan.", "Tanlangan xona shu vaqtda band.", "Javob yozing yoki kamida bitta fayl biriktiring."

11. FRONTENDDAGI MOCK -> BACKEND XARITASI

Hozir ma'lumot brauzerda quyidagi kalitlarda turibdi (localStorage). Backend ulanganda bular API ga almashadi:

```
skillsup:session       login holati (rol)                      -> sessiya tokeni (sessions jadvali)
skillsup:preferences   mavzu va bildirishnoma sozlamalari      -> user_settings
skillsup:lessons       o'quvchining darslari, progress, vazifa  -> lessons, lesson_progress, student_homework, submissions
skillsup:teacher       ustozning darslari, boshqa o'quvchilar   -> lessons, homeworks, attendance, student_homework
skillsup:exams         imtihon natijalari                      -> exam_results
skillsup:admin         o'quvchilar, guruhlar, xonalar, ustozlar, parol xeshlari, to'lov rejalari, eslatmalar
                                                              -> users, groups, group_schedule, group_members, rooms, payment_plans, payment_notices, notifications
```

Mock fayllar (src/data): groups.ts, lessons.ts, teacher.ts, students.ts, leaderboard.ts, payments.ts, notifications.ts, guides.ts, career.ts, analytics.ts, admin.ts. Ular seed (boshlang'ich ma'lumot) uchun namuna bo'la oladi.

12. SEED (BOSHLANG'ICH) MA'LUMOT

Demo uchun frontenddagi ma'lumotlar:

- Guruhlar: gr1 "Fullstack dasturlash" (ustoz t1, Du/Ch 18:00-20:00 A xona, Ju 17:00-18:00 onlayn), gr2 "UI/UX dizayn" (ustoz t2, Se/Pa 18:00-20:00 Dizayn studiyasi), gr3 "Backend asoslari" (ustoz t1, Se 18:00-20:00 B xona, Sh 10:00-12:00 B xona).
- Xonalar: A xona (12), B xona (12), C xona (10), Dizayn studiyasi (10).
- Ustozlar: Dilnoza Rahimova (t1), Gulnora Saidova (t2), Jasur Karimov (t3).
- Demo hisoblar (faqat ishlab chiqish muhitida!): admin / admin123, ustoz dilnoza.rahimova / ustoz123, o'quvchi bekzod.rahimov / oquvchi123. Ishlab chiqarishda bu parollarni ishlatmang va majburan almashtiring.
- To'lov rejalari: gr1 500000 (5-dekabr), gr2 400000 (7-dekabr), gr3 450000 (10-dekabr).

13. OCHIQ SAVOLLAR (backend bilan kelishish kerak)

1. To'lovlarni kim yozadi: admin qo'lda kiritadimi yoki to'lov tizimi (Click, Payme, Uzcard) bilan ulanadimi? Hozir frontendda faqat to'lov tarixi ko'rinadi va admin eslatma yuboradi.

2. Bildirishnomalar real vaqtda (WebSocket/SSE) kelsinmi yoki sahifa ochilganda olinadimi? Email/push yuborish kerakmi?

3. Kunlik seriya (streak) qaysi harakatlarga qarab hisoblanadi va yarim tunda qanday uziladi?

4. Tahlildagi "ko'nikmalar" (skills) foizi qanday hisoblanadi? Darsga mavzu/teg qo'shish kerakmi?

5. Reytingdagi delta (haftalik o'zgarish) uchun haftalik snapshot kerakmi?

6. O'quvchi guruhdan chiqarilsa yoki guruh yopilsa nima bo'ladi (ma'lumot saqlanadimi)? Hozir frontendda chiqarish yo'q.

7. Bir nechta admin va ularning harakat jurnali (audit log) kerakmi?

8. Dars boshlangandan keyin o'chirish/tahrirlash (nomini o'zgartirish, imtihon belgisini olish) kerakmi? Hozir faqat tavsif, video va davomiylik o'zgaradi.

9. Parolni unutgan foydalanuvchi uchun tiklash (admin tomonidan yangi parol berish yetarlimi)?

10. Fayllar qancha vaqt saqlanadi va kim yuklab olishi mumkin?

14. XAVFSIZLIK TALABLARI

- Parollar faqat argon2id/bcrypt xesh ko'rinishida; hech qayerda log yoki javobda ochiq chiqmasin.
- Har endpointda rol va egalik tekshiruvi (ustoz faqat o'z guruhi, o'quvchi faqat o'z ma'lumoti).
- Login endpointiga urinishlarni cheklash, umumiy xato matni.
- Fayl yuklashda kengaytma, mime, hajm, nom tekshiruvi; fayllarni veb-ildizdan tashqarida saqlash; yuklab olishda ruxsatni tekshirish.
- CORS faqat frontend domeniga; HTTPS; sessiya tokenlari uchun qisqa muddat (SESSION_HOURS); token bazada faqat xesh ko'rinishida.
- Admin harakatlari (login/parol o'zgartirish, o'quvchi qo'shish, to'lov eslatmasi) jurnalga yozilsin (tavsiya).
- SQL in'ektsiyadan himoya (parametrlangan so'rovlar), kiruvchi matnlarni tozalash.

15. NESTJS UCHUN TUZILMA

Yuqoridagi jadvallar, qoidalar va endpointlar NestJS da quyidagicha joylashtirilishi tavsiya etiladi.

15.1 Papka va modullar

```
src/
  main.ts                     ValidationPipe (whitelist, transform), global prefiks /api/v1, CORS, Swagger
  app.module.ts
  config/                     ConfigModule: konstantalar (6.1-bo'lim), env, DB
  common/
    decorators/               @Roles(...), @CurrentUser()
    guards/                   SessionAuthGuard, RolesGuard
    filters/                  HttpExceptionFilter (10-bo'limdagi xato formati)
    pipes/                    fayl tekshiruvi (kengaytma, hajm)
    enums/                    Role, HomeworkStatus, Day
  auth/                       login, logout, refresh, SessionsService (sessiya tokenlari), parol xeshlash (argon2/bcrypt)
  users/                      User entity, /me, /me/password, /me/settings
  files/                      Multer yuklash, yuklab olish (Range qo'llab-quvvatlash: video uchun)
  groups/                     Group, GroupSchedule, GroupMember
  rooms/                      Room, bandlik va bo'sh xonalar hisobi
  lessons/                    Lesson, LessonProgress, Attendance
  homework/                   Homework, StudentHomework, Submission (+ fayllar)
  exams/                      imtihon qamrovi, ruxsat, ExamResult
  rewards/                    XP, kumush, bosqich, reyting hisoblari (7-bo'lim); faqat servis, controller kerak emas
  payments/                   Payment, PaymentPlan, PaymentNotice
  notifications/              Notification
  content/                    CareerVideo, Guide, Faq
  student/                    o'quvchi panel controllerlari (9.2)
  teacher/                    ustoz panel controllerlari (9.3)
  admin/                      admin panel controllerlari (9.4)
```

Tavsiya: controllerlar rol bo'yicha (student/, teacher/, admin/ modullari) va ular domen modullari servislarini chaqiradi. Shunda endpoint prefikslari (/student/*, /teacher/*, /admin/*) rolga aniq mos tushadi.

15.2 Guard va dekoratorlar (rol guard)

```
@Controller('admin')
@UseGuards(SessionAuthGuard, RolesGuard)
@Roles(Role.ADMIN)
export class AdminStudentsController { ... }

@Controller('teacher')
@UseGuards(SessionAuthGuard, RolesGuard)
@Roles(Role.TEACHER)
export class TeacherLessonsController {
  // ustozning o'z guruhi ekanini servisda tekshiring: group.teacherId === user.id, aks holda ForbiddenException
}

@Controller('student')
@UseGuards(SessionAuthGuard, RolesGuard)
@Roles(Role.STUDENT)
export class StudentLessonsController {
  // o'quvchi id sini HECH QACHON URL/body dan olmang: @CurrentUser() user.id
}
```

- SessionAuthGuard: tokenni sessions jadvalidan tekshiradi (xesh bo'yicha), req.user = { id, role, sessionId }. Yaroqsiz yoki muddati o'tgan bo'lsa 401.
- RolesGuard: Reflector orqali @Roles ni o'qiydi, req.user.role mos kelmasa 403 (4.2-bo'lim).
- Login endpointi @Public() bilan belgilanadi (global SessionAuthGuard bo'lsa).
- Egalik tekshiruvi (ustoz-guruh, o'quvchi-o'z ma'lumoti) servis qatlamida bajariladi.

15.3 DTO va validatsiya (class-validator)

Har so'rov uchun DTO. Qoidalar 10-bo'limdagi jadval bilan bir xil:

```
LoginDto            @IsString() login; @IsString() password
CreateStudentDto    @Matches(/^\S+\s+\S+/) fullName; @Matches(/^\+998[ -]?\d{2}[ -]?\d{3}[ -]?\d{2}[ -]?\d{2}$/) phone;
                    @Matches(/^[a-z0-9._-]{4,30}$/) login; @MinLength(8) password; @IsArray() @IsOptional() groupIds
CreateRoomDto       @MinLength(2) name; @IsInt() @Min(1) @Max(60) capacity
CreateGroupDto      @MinLength(3) name; @MinLength(3) course; teacherId; @ArrayMinSize(1) @IsEnum(Day, {each:true}) days;
                    @IsIn(['08:00','10:00','12:00','14:00','16:00','18:00','20:00']) start; roomId;
                    @IsInt() @Min(10000) amount; @IsDateString() dueDate (bugundan oldin emas: custom validator)
StartLessonDto      groupId; @MinLength(3) title; @IsBoolean() @IsOptional() isExam
UpdateLessonInfoDto @IsOptional() description (bo'sh yoki >= 10); @Matches(/^https?:\/\/\S+$/) videoUrl; videoFileId; @IsInt() @Min(10) @Max(240) durationMin
AssignHomeworkDto   @MinLength(10) task; @ArrayMinSize(1) steps; @IsIn([1,3,24,72,168]) limitHours; @ArrayMaxSize(5) attachmentIds
SubmitHomeworkDto   @MaxLength(2000) text; @ArrayMaxSize(3) fileIds   (text yoki fayl: kamida bittasi: servisda tekshiring)
GradeDto            @IsInt() @Min(1) @Max(100) ball
ReturnDto           @IsString() @MinLength(1) feedback
PaymentPlanDto      @IsInt() @Min(10000) amount; @IsDateString() dueDate
```

Global: app.useGlobalPipes(new ValidationPipe({ whitelist: true, forbidNonWhitelisted: true, transform: true })). Xatolar HttpExceptionFilter orqali 10-bo'limdagi formatga o'tkaziladi ({ error: { code, message, fields } }).

15.4 Entity lar (TypeORM nomlari, 5-bo'limdagi jadvallar bilan 1:1)

User, Student, Teacher, Room, Group, GroupSchedule, GroupMember, Lesson, LessonProgress, Attendance, Homework, HomeworkAttachment, StudentHomework, Submission, SubmissionFile, ExamResult, Invoice, Payment, PaymentPlan, PaymentNotice, Notification, UserSettings, StoredFile (files jadvali), CareerVideo, Guide, Faq, Activity.

Muhim cheklovlar entity/migratsiyada:

- User.login unique (kichik harfga keltirib saqlang).
- GroupSchedule: unique (roomId, day, startTime) (roomId null bo'lganda cheklov yo'q).
- Homework.lessonId unique; StudentHomework PK (homeworkId, studentId).
- LessonProgress.percent faqat oshadi: servisda Math.max(eski, yangi).
- Parol xeshi @Column({ select: false }) bo'lsin: tasodifan javobga chiqib ketmasin.

15.5 Servislar (biznes mantiq shu yerda)

```
RewardsService      attended, ball, kechikish jarimasi, XP/kumush, level, guruh reytingi (7.1-7.4)
ExamsService        imtihon qamrovi (7.5), ruxsat (missing darslar), ball kiritish (blocked bo'lsa ConflictException)
AttendanceService   belgilash, mark-all, save (qulflash; hamma o'quvchi uchun aniq yozuv), reopen; saqlangan bo'lsa ConflictException
HomeworkService     berish/yangilash (faqat NOT_DONE va berilmaganlarga), topshirish, baholash, qaytarish (+24 soat)
ScheduleService     xona bandligi, bo'sh xonalar, ustoz to'qnashuvi (6.7); guruh ochish tranzaksiyada
ReportsService      hisobot: filtrlar, summary, oldingi davr bilan solishtirish, grafik seriyasi, Excel eksport
PaymentsService     hisob yaratish, hisobga to'lov qo'shish; rejani yangilash + PaymentNotice + guruhdagi hamma o'quvchiga Notification (bitta tranzaksiyada)
AccountsService     admin: login/parol o'zgartirish; o'zi: parolni o'zgartirish
```

Ko'p jadvalga yoziladigan amallar (guruh ochish, to'lov eslatmasi, davomatni saqlash, vazifa berish) TypeORM transaction yoki Prisma $transaction ichida bajarilsin.

15.6 Boshqa NestJS tavsiyalari

- Konfiguratsiya: @nestjs/config; konstantalar (ATTENDANCE_XP, LATE_PENALTY va h.k.) bitta config faylida (6.1-bo'lim).
- Fayl yuklash: @nestjs/platform-express (Multer) + o'z FileValidationPipe (8-bo'limdagi jadval). Videoni StreamableFile bilan Range so'rovlarini qo'llab-quvvatlab bering.
- Login himoyasi: @nestjs/throttler (login endpointiga qattiq limit).
- Hujjatlash: @nestjs/swagger, shu hujjatdagi endpointlar bilan bir xil.
- Migratsiyalar va seed: demo ma'lumot (12-bo'lim) faqat development muhitida.
- Testlar: RewardsService va ExamsService uchun unit testlar (formulalar 7-bo'limdagi misollar bilan).
- Tur nomlari: frontend src/types/models.ts dagi nomlar (Lesson, Homework, Group, ...) bilan mos tursa, ulash osonlashadi.

Hujjat Skills Up frontend kodi asosida tuzildi. Savol yoki kelishmovchilik bo'lsa frontend dasturchi bilan aniqlashtiring.